Machine Learning

Foundations, Algorithms, and the Practitioner's Workflow

This guide covers the conceptual foundations of machine learning — how models learn from data, the major families of algorithms, how to evaluate whether a model actually works, and the practical workflow that turns a dataset into a deployed system. It is aimed at readers who want a solid working mental model rather than a purely mathematical treatment.

1. What Machine Learning Is

Machine learning is the practice of building systems that improve their performance on a task by learning patterns from data, rather than following rules that a programmer writes explicitly. Instead of hand-coding "if the email contains these words, mark it as spam," a machine learning system is shown thousands of examples of spam and non-spam email and learns, statistically, which patterns are predictive.

Every ML system has three ingredients: data to learn from, a model with adjustable parameters, and a loss function that measures how wrong the model's current predictions are. Training is the process of adjusting the parameters to reduce that loss, typically through an optimization algorithm such as gradient descent.

- Supervised learning — learns from labeled examples (input, correct output) pairs.
- Unsupervised learning — finds structure in unlabeled data, e.g. clustering or dimensionality reduction.
- Reinforcement learning — learns by trial and error, guided by rewards from an environment.
- Semi-supervised and self-supervised learning — use a small labeled set plus a much larger unlabeled set.

2. The Core Algorithm Families

2.1 Linear and Logistic Regression

Linear regression predicts a continuous value as a weighted sum of input features. Logistic regression adapts the same idea for classification by passing that weighted sum through a sigmoid function to produce a probability. Both remain widely used because they're fast, interpretable, and a strong baseline against which more complex models are compared.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
```

Figure 2.1 — Baseline classifier with scikit-learn

2.2 Tree-Based Models

Decision trees split the data recursively along feature thresholds to separate classes or reduce prediction error. On their own they overfit easily, so in practice they are almost always used as an

ensemble: random forests average many trees trained on bootstrapped samples, while gradient boosting (XGBoost, LightGBM, CatBoost) builds trees sequentially, each one correcting the errors of the ones before it. Gradient-boosted trees remain the strongest general-purpose approach for structured, tabular data.

2.3 Support Vector Machines and k-NN

Support vector machines find the boundary that maximizes the margin between classes, optionally using a kernel trick to handle non-linear boundaries. k-Nearest Neighbors is even simpler: it classifies a new point by looking at the majority label among its closest neighbors in the training set, with no explicit training phase at all.

2.4 Unsupervised Methods

k-means clustering groups points into k clusters by minimizing within-cluster distance. Principal Component Analysis (PCA) reduces the dimensionality of data by projecting it onto the directions of greatest variance, which is useful both for visualization and as a preprocessing step before other algorithms.

3. The Practitioner's Workflow

Building a model is a small part of a real ML project. Most of the effort goes into understanding and preparing the data, and most of the risk comes from evaluating the model incorrectly rather than from picking the wrong algorithm.

Stage	What Happens
Problem framing	Define the target variable and what success looks like
Data collection	Gather and consolidate data from source systems
Exploratory analysis	Understand distributions, missing values, correlations
Feature engineering	Create and transform inputs the model will learn from
Model training	Fit one or more candidate models to the training data
Evaluation	Measure performance on held-out data with the right metric
Deployment	Serve the model behind an API or batch pipeline
Monitoring	Watch for data drift and performance decay over time

Table 3.1 — A typical end-to-end machine learning workflow

3.1 Feature Engineering

Raw data is rarely in a form a model can use directly. Feature engineering covers encoding categorical variables, scaling numerical ones, handling missing values, and constructing new features that make patterns easier for the model to find — such as turning a timestamp into day-of-week and hour-of-day.

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

preprocess = ColumnTransformer([
    ("num", StandardScaler(), ["age", "income"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["city", "plan_type"]),
])

pipeline = Pipeline([
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=1000)),
])
pipeline.fit(X_train, y_train)
```

Figure 3.1 — A preprocessing + model pipeline

3.2 Evaluating Models Correctly

The single most common mistake in applied ML is evaluating a model on data it has already seen, which produces an overly optimistic estimate of real-world performance. Cross-validation, a held-out test set, and — for time-series data — evaluation that respects chronological order are all ways of guarding against this.

- Accuracy — fine for balanced classes, misleading when classes are imbalanced.
- Precision and recall — trade off false positives against false negatives.
- F1 score — the harmonic mean of precision and recall, useful for a single summary number.
- ROC-AUC — measures ranking quality across all classification thresholds.
- RMSE / MAE — standard error metrics for regression problems.

4. Overfitting, Underfitting, and Regularization

A model that is too flexible for the amount of training data will memorize noise rather than learning the underlying pattern — this is overfitting, and it shows up as strong training performance but poor performance on new data. A model that is too simple will underfit, performing poorly on both. Regularization techniques such as L1/L2 penalties, dropout in neural networks, and early stopping all work by constraining the model's effective complexity so it generalizes better.

- L2 regularization (ridge) shrinks weights toward zero smoothly.
- L1 regularization (lasso) can drive some weights exactly to zero, performing feature selection.
- Cross-validation helps detect overfitting before it reaches production.
- More training data is often the single most effective cure for overfitting.

5. Where Machine Learning Is Applied

- Fraud detection and credit risk scoring in financial services.
- Recommendation systems for e-commerce, streaming, and social platforms.
- Predictive maintenance in manufacturing using sensor data.
- Medical diagnosis support from imaging and clinical records.
- Demand forecasting and inventory optimization in retail and logistics.
- Search ranking and ad targeting across nearly every major internet platform.

6. Closing Thoughts

Classical machine learning remains the workhorse of most real-world data problems, especially on structured, tabular data where gradient-boosted trees and well-engineered features consistently outperform more exotic approaches. Understanding these fundamentals — how models learn, how to evaluate them honestly, and how to avoid the common pitfalls of overfitting and data leakage — is the foundation on which deep learning and generative AI, covered in the companion guides, are built.